
1. Configuração do Projeto e Instalação

Primeiro, vamos iniciar o projeto NestJS e instalar as dependências do Prisma.

Bash

```
# Instalar Nest CLI (se não tiver) e criar projeto
npm i -g @nestjs/cli
nest new project-name

# Entrar na pasta e instalar dependências
cd project-name
npm install -D prisma
npm install @prisma/client
npm i @prisma/adapters-pg
npm i -D dotenv
npm install class-validator class-transformer
npm install --save @nestjs/swagger
```

Listando os pacotes instalados no projeto NestJS:

```
npm list
```

Inicialize o Prisma para criar a pasta `prisma` e o arquivo `.env`:

```
npx prisma init
```

2. Modelagem do Banco de Dados

Abra o arquivo `prisma/schema.prisma` e defina a tabela de usuários:

```
generator client {
  provider = "prisma-client"
  output = "../src/generated/prisma"
  moduleFormat = "cjs"
}

datasource db {
```

```
    provider = "postgresql"
  }

  model User {
    id Int @id @default(autoincrement())
    email String @unique
    name String?
    password String
    createdAt DateTime @default(now())
    updatedAt DateTime @updatedAt()
  }
```

Agora, abra `.env` e ajuste a `DATABASE_URL` variável de ambiente para que fique da seguinte forma:

```
DATABASE_URL="postgresql://USER:PASSWORD@HOST:PORT/DATABASE?
schema=SCHEMA"
```

```
DATABASE_URL="postgresql://postgres:postgres@localhost:5432/DBdev?
schema=public"
```

Configure o seu `.env` com as credenciais do seu PostgreSQL e execute a migration:

Bash

```
npx prisma migrate dev --name init
npx prisma generate
```

3. Configurando o Prisma Service

Para que o NestJS gerencie a conexão com o banco, crie um serviço dedicado:

Bash

```
nest generate module prisma
nest generate service prisma
```

No `prisma.service.ts`:

TypeScript

```
import 'dotenv/config';
import { Injectable } from '@nestjs/common';
import { PrismaPg } from '@prisma/adapters-pg';
import { PrismaClient } from '../generated/prisma/client';

@Injectable()
export class PrismaService extends PrismaClient {
  constructor() {
    const databaseUrl = process.env.DATABASE_URL;
    if (!databaseUrl) throw new Error('DATABASE_URL não definida');
    // Em algumas versões é "connectionString" em vez de "url".
    const adapter = new PrismaPg({ connectionString: databaseUrl });
    super({ adapter });
  }
}
```

4. Gerando o Recurso de Usuário (CRUD)

Utilizaremos o gerador automático do NestJS para criar a estrutura base:

Bash

```
nest generate resource users
# Selecione "REST API" e "Yes" para gerar os entry points do CRUD.
```

Observação:

- atualizar o arquivo '**prisma.module.ts**' adicionando exports: [PrismaService]

```
import { Module } from '@nestjs/common';
import { PrismaService } from '../prisma.service';

@Module({
  providers: [PrismaService],
  exports: [PrismaService], // expor o PrismaService para outros módulos
})

export class PrismaModule {}
```

- atualizar o arquivo '**users.module.ts**' adicionando imports: [PrismaModule]

```
import { Module } from '@nestjs/common';
import { PrismaModule } from '../prisma/prisma.module';
import { UsersService } from './users.service';
import { UsersController } from './users.controller';

@Module({
  imports: [PrismaModule],          // importado o modulo PrismaModule
  controllers: [UsersController],
  providers: [UsersService],
})
export class UsersModule {}
```

Definindo os DTOs (Data Transfer Objects)

Instale o `class-validator` para validar os dados de entrada:

Bash

```
npm install class-validator class-transformer
```

No arquivo `src/users/dto/create-user.dto.ts`:

TypeScript

```
import { IsEmail, IsNotEmpty, IsString, MinLength } from 'class-validator';

export class CreateUserDto {
  @IsEmail()
  email: string;

  @IsString()
  @IsNotEmpty()
  name: string;

  @IsString()
  @MinLength(6)
  password: string;
}
```

5. Implementando a Lógica no Service

No `src/users/users.service.ts`, injete o `PrismaService` e crie os métodos:

TypeScript

```
import { Injectable } from '@nestjs/common';
import { PrismaService } from '../../prisma/prisma.service';
import { CreateUserDto } from '../../dto/create-user.dto';
import { UpdateUserDto } from '../../dto/update-user.dto';

@Injectable()
export class UsersService {
  constructor(private prisma: PrismaService) {}

  create(createUserDto: CreateUserDto) {
    return this.prisma.user.create({ data: createUserDto });
  }

  findAll() {
    return this.prisma.user.findMany();
  }

  findOne(id: number) {
    return this.prisma.user.findUnique({ where: { id } });
  }

  update(id: number, updateUserDto: UpdateUserDto) {
    return this.prisma.user.update({
      where: { id },
      data: updateUserDto,
    });
  }

  remove(id: number) {
    return this.prisma.user.delete({ where: { id } });
  }
}
```

6. Ajustando o Controller e Ativando Validação

Certifique-se de que o `UsersController` está passando os parâmetros corretamente (converta IDs para `number`).

No seu `main.ts`, ative o `ValidationPipe` global:

TypeScript

```
import { ValidationPipe } from '@nestjs/common';
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  app.useGlobalPipes(new ValidationPipe()); // Ativa validação dos DTOs
  await app.listen(3000);
}
bootstrap();
```

Resumo do Fluxo de Dados

Camada	Responsabilidade
DTO	Define a forma e as regras de validação do dado que entra.
Controller	Lida com as rotas HTTP e extrai parâmetros.
Service	Contém a regra de negócio e chama o Prisma.
Prisma	Realiza a comunicação direta com o PostgreSQL.

1. Instalação das Dependências

Instale o módulo do Swagger para NestJS:

Bash

```
npm install --save @nestjs/swagger
```

2. Configuração no `main.ts`

Você precisa inicializar o Swagger no arquivo de entrada da sua aplicação.

TypeScript

```

import { NestFactory } from '@nestjs/core';
import { SwaggerModule, DocumentBuilder } from '@nestjs/swagger';
import { AppModule } from './app.module';
import { ValidationPipe } from '@nestjs/common';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);

  // Configuração do ValidationPipe (importante para o Swagger ler as
  // validações)
  app.useGlobalPipes(new ValidationPipe());

  // Configuração do Swagger
  const config = new DocumentBuilder()
    .setTitle('User CRUD API')
    .setDescription('Documentação da API de Usuários com NestJS e Prisma')
    .setVersion('1.0')
    .addTag('users')
    .build();

  const document = SwaggerModule.createDocument(app, config);
  SwaggerModule.setup('api', app, document); // Rota onde o Swagger estará
  // disponível

  await app.listen(3000);
  console.log(`Application is running on: http://localhost:3000/api`);
}
bootstrap();

```

3. Decorando os DTOs

Para que o Swagger saiba quais campos são esperados e quais são obrigatórios, usamos decorators nos nossos DTOs. Isso gera o esquema visual na documentação.

No arquivo `src/users/dto/create-user.dto.ts`:

TypeScript

```

import { ApiProperty } from '@nestjs/swagger';
import { IsEmail, IsNotEmpty, IsString, MinLength } from 'class-validator';

export class CreateUserDto {
  @ApiProperty({ example: 'joao@email.com', description: 'O email do

```

```

usuário' })
  @IsEmail()
  email: string;

  @ApiProperty({ example: 'João Silva', description: 'Nome completo' })
  @IsString()
  @IsNotEmpty()
  name: string;

  @ApiProperty({ example: 'senha123', description: 'Senha com no mínimo 6
caracteres', minLength: 6 })
  @IsString()
  @MinLength(6)
  password: string;
}

```

4. Decorando o Controller

Agora, vamos adicionar informações sobre as respostas (sucesso, erro, etc.) no `src/users/users.controller.ts`.

TypeScript

```

import { Controller, Get, Post, Body, Patch, Param, Delete } from
 '@nestjs/common';
import { UsersService } from './users.service';
import { CreateUserDto } from './dto/create-user.dto';
import { UpdateUserDto } from './dto/update-user.dto';
import { ApiTags, ApiOperation, ApiResponse } from '@nestjs/swagger';

@ApiTags('users') // Agrupa os endpoints sob a tag 'users'
@Controller('users')
export class UsersController {
  constructor(private readonly usersService: UsersService) {}

  @Post()
  @ApiOperation({ summary: 'Criar um novo usuário' })
  @ApiResponse({ status: 201, description: 'Usuário criado com sucesso.' })
  @ApiResponse({ status: 400, description: 'Dados inválidos.' })
  create(@Body() createUserDto: CreateUserDto) {
    return this.usersService.create(createUserDto);
  }
}

```

```
@Get()
@ApiOperation({ summary: 'Listar todos os usuários' })
findAll() {
  return this.usersService.findAll();
}

@Get('/:id')
@ApiOperation({ summary: 'Buscar um usuário pelo ID' })
findOne(@Param('id') id: string) {
  return this.usersService.findOne(+id);
}

@Patch('/:id')
@ApiOperation({ summary: 'Atualizar um usuário' })
update(@Param('id') id: string, @Body() updateUserDto: UpdateUserDto) {
  return this.usersService.update(+id, updateUserDto);
}

@Delete('/:id')
@ApiOperation({ summary: 'Remover um usuário' })
remove(@Param('id') id: string) {
  return this.usersService.remove(+id);
}
}
```

5. Visualizando o Resultado

1. Inicie sua aplicação: `npm run start:dev`
2. Acesse no navegador: `http://localhost:3000/api`

Você verá uma interface amigável listando todos os seus métodos HTTP. Você pode clicar em **"Try it out"**, preencher o JSON e executar a chamada real para o seu banco PostgreSQL.

Dica de Especialista: CLI Plugin

Para evitar ter que colocar `@ApiProperty` em todos os campos manualmente, você pode habilitar o **Swagger Plugin** no arquivo `nest-cli.json`:

JSON

```
{
  "collection": "@nestjs/schematics",
  "sourceRoot": "src",
  "compilerOptions": {
    "plugins": ["@nestjs/swagger"]
  }
}
```

Isso fará com que o NestJS infira as propriedades do Swagger automaticamente a partir dos tipos do TypeScript.

Gostaria que eu te mostrasse como tratar os erros do Prisma (como email duplicado) para que a API retorne um erro amigável em vez de um erro 500?